



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Летучка № 10

по курсу «Языки и методы программирования»

«ТСР-клиент с GUI и обработкой исключений»

Студент группы ИУ9-22Б
Булдаков А. С.

Преподаватель
Посевин Д. П.

Москва 2026

Содержание

Цель работы	1
Класс ConnectionHandler	1
Интерфейс NetworkListener	2
Метод connect()	2
Handshake	3
Исключения	4
Базовый класс ConnectionException	4
HandshakeException	4
ConnectionLostException	4
Прослушивание сообщений	5
GUI-клиент	6
Темная тема (FlatLaF)	8
Вывод	8

Цель работы

Разработать TCP-клиент с графическим интерфейсом на Swing и обработкой исключений. Реализовать механизм переподключения (reconnect) и протокол handshake.

Задачи:

- Реализовать GUI-клиент на Swing.
- Обработать исключения при подключении.
- Реализовать handshake между клиентом и сервером.
- Добавить автоматическое переподключение.

Класс ConnectionHandler

Класс для управления TCP-соединением.

Интерфейс NetworkListener

Client.java

```
class ConnectionHandler {
    private static final String DEFAULT_HOST = "185.104.251.226";
    private static final int DEFAULT_PORT = 3896;

    public interface NetworkListener {
        void onMessage(String msg);
        void onLog(String symbol, String msg);
        void onReady();
    }

    private Socket socket;
    private PrintWriter out;
    private BufferedReader in;
    private NetworkListener listener;
}
```

Интерфейс NetworkListener обеспечивает обратный вызов для событий сети.

Метод connect()

Client.java

```
public void connect(String host, int port) {
    new Thread(() → {
        try {
            attempt(host, port);
        } catch (ConnectionException e) {
            listener.onLog("/!\\", "Ошибка: " + e.getMessage() + ".  
Реконект к дефолту ... ");
            try {
                attempt(DEFAULT_HOST, DEFAULT_PORT);
            } catch (ConnectionException ex) {
                listener.onLog("[!]", "Фатальная ошибка: " +  
ex.getMessage());
            }
        }
    }).start();
}
```

Метод connect() запускает подключение в отдельном потоке и при ошибке выполняет переподключение к хосту по умолчанию.

Handshake

Client.java

```
private void attempt(String host, int port) throws
ConnectionException {
    listener.onLog("[*]", "Подключение к " + host + ":" + port);
    try {
        socket = new Socket();
        socket.connect(new InetSocketAddress(host, port), 3000);

        out = new PrintWriter(new BufferedWriter(new
OutputStreamWriter(socket.getOutputStream())), true);
        in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));

        out.println("SYN");
        String synAck = in.readLine();
        if (synAck != null && synAck.contains("SYN-ACK")) {
            listener.onLog("[+]", "Handshake успешен");
            listener.onReady();
            startListening();
        } else {
            throw new HandshakeException("Handshake failed: " + synAck);
        }
    } catch (IOException e) {
        throw new ConnectionException("Не удалось подключиться к " +
host + ":" + port, e);
    }
}
```

Протокол handshake:

1. Клиент отправляет SYN
2. Сервер отвечает SYN-ACK
3. При успехе запускается прослушивание
4. При ошибке — исключение

Исключения

Базовый класс ConnectionException

ConnectionException.java

```
public class ConnectionException extends Exception {  
    public ConnectionException(String message) {  
        super(message);  
    }  
  
    public ConnectionException(String message, Throwable cause) {  
        super(message, cause);  
    }  
}
```

HandshakeException

HandshakeException.java

```
public class HandshakeException extends ConnectionException {  
    public HandshakeException(String message) {  
        super(message);  
    }  
}
```

ConnectionLostException

ConnectionLostException.java

```
public class ConnectionLostException extends RuntimeException {  
    public ConnectionLostException(String message) {  
        super(message);  
    }  
}
```

Прослушивание сообщений

Client.java

```
private void startListening() {
    new Thread(() -> {
        try {
            String line;
            while ((line = in.readLine()) != null) {
                if (line.equals("FLOW_STOPPED")) {
                    listener.onLog("/!\\", "Сервер остановил поток.  
Перезапуск flow ... ");
                    sendFlowCommand();
                } else {
                    listener.onMessage(line);
                }
            }
        } catch (IOException e) {
            listener.onLog("[!]", "Связь разорвана: " + e.getMessage());
        }
    }).start();
}
```

Метод startListening() в отдельном потоке ожидает сообщения от сервера. При получении FLOW_STOPPED автоматически отправляется команда flow.

GUI-клиент

Client.java

```

public class Client extends JFrame implements
ConnectionHandler.NetworkListener {
    private JTextField serverInput, portInput, messageInput;
    private JTextArea logArea;
    private JButton connectButton, sendButton;
    private ConnectionHandler handler;

    public Client() {
        setTitle("TCP Flow Client");
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setSize(500, 500);
        handler = new ConnectionHandler(this);
        initComponents();
    }

    private void initComponents() {
        JPanel top = new JPanel(new GridLayout(1, 5, 5, 5));
        serverInput = new JTextField("185.104.251.226");
        portInput = new JTextField("3896");
        connectButton = new JButton("Connect");
        connectButton.addActionListener(e → handler.connect(...));

        logArea = new JTextArea();
        logArea.setEditable(false);
        logArea.setBackground(new Color(30, 30, 30));
        logArea.setForeground(new Color(200, 200, 200));

        messageInput = new JTextField();
        sendButton = new JButton("Send");
        sendButton.addActionListener(e → {
            if (messageInput.getText().equals("flow"))
                handler.sendFlowCommand();
            else
                handler.sendMessage(messageInput.getText());
            messageInput.setText("");
        });
    }

    @Override
    public void onLog(String symbol, String msg) {
        SwingUtilities.invokeLater(() → {
            logArea.append(symbol + " " + msg + "\n");
            logArea.setCaretPosition(logArea.getDocument().getLength());
        });
    }

    @Override
    public void onMessage(String msg) {
        SwingUtilities.invokeLater(() → {

```


Компоненты GUI:

- serverInput — поле ввода IP-адреса
- portInput — поле ввода порта
- connectButton — кнопка подключения
- logArea — область вывода сообщений
- messageInput — поле ввода сообщения
- sendButton — кнопка отправки

Темная тема (FlatLaF)

Client.java

```
public static void main(String[] args) {  
    FlatDarkLaf.setup();  
    SwingUtilities.invokeLater(() -> new Client().setVisible(true));  
}
```

Используется FlatDarkLaf для современного темного оформления.

Вывод

В ходе лабораторной работы были реализованы:

1. ConnectionHandler — класс управления соединением с переподключением
2. Handshake — протокол синхронизации между клиентом и сервером
3. Исключения — ConnectionException, HandshakeException, ConnectionLostException
4. GUI на Swing — клиент с темной темой
5. Многопоточность — connect и listen в отдельных потоках

Клиент автоматически обрабатывает разрыв соединения и выполняет переподключение.